University *of Ljubljana*
Faculty *of Computer and Information Science*

Travel Planning Chatbot

Nina Kokalj and Tia Šarkezi

Abstract

This project presents a travel chatbot that combines a large language model with tool-using agents to support itinerary generation and real-time travel assistance. The system integrates external tools for weather forecasting and train connection search, enabling access to up-to-date information beyond the capabilities of a standalone language model. The initial base model demonstrated strong performance in generating itineraries and travel recommendations but lacked factual reliability due to the absence of live data. To address this, a smolagents-based architecture was introduced to enable structured tool usage through API calls. Further improvements were achieved through prompt engineering, few-shot examples, and simplified tool schemas, which improved output consistency and reduced parsing errors. Evaluation shows that while tool augmentation improves correctness, it can negatively affect creative output without careful prompting and routing design. The results highlight a trade-off between creative generation and grounded, real-time information retrieval, emphasizing the importance of prompt design, schema structure, and agent coordination in building reliable LLM-based travel systems.

Keywords

Travel chatbot, Large language model, Tool-using agents, API integration, Prompt Engineerings

Advisors: Aleš Žagar

Introduction

This project focuses on developing a travel chatbot that provides itinerary generation, travel recommendations, live weather information, and train schedule information in a single system. The project combines large language models with external tools to extend chatbot capabilities beyond text generation. The main objective is to create a system that generates useful travel plans while retrieving accurate real-time information. The main challenge is balancing flexible itinerary generation with reliable external data retrieval.

Related work

Studies [1, 2, 3] demonstrate that while LLMs excel at dialogue, they struggle with logistical ”hard constraints” like budgets and schedules. To solve this, researchers are now using hybrid architectures that pair LLMs with deterministic filtering and scoring systems. These hybrid models have achieved up to 91% recommendation accuracy, significantly reducing decision fatigue for the 40% of global travelers who now use AI as their primary trip-discovery tool. By integrating user-specific weights for travel style and budget, these systems deliver high-fidelity itineraries that mirror the expertise of a human travel consultant.

Methods

The system is built as a pipeline that combines a base **large language model** with a **tool-using agent** layer. User queries are first processed by the LLM, which either generates a direct response or decides to invoke external tools through an agent framework based on `smolagents`. In the base configuration, the model is responsible for itinerary generation, travel recommendations, and conversational responses without access to external data.

In the extended system, an agent layer enables interaction with external tools, including a weather forecasting tool and a train connection search tool. The agent follows a reasoning loop where it decides when to call a tool, executes the call, and integrates the returned observation into the final response. The system is guided by a structured **system prompt** that defines rules for itinerary generation, tool usage, and final output formatting. Few-shot examples are included to enforce correct execution patterns and reduce formatting errors. Additionally, tool schemas were simplified from nested JSON structures to flatter formats to improve parsing reliability and reduce model error rates.

Experiments

Base Model Performance

The initial version of the travel chatbot was developed using only a base large language model, **Llama-3.1-8B-Instruct** [4], without external tool integration. The model demonstrated strong reasoning and instruction-following capabilities directly out-of-the-box. It performed well in generating detailed travel itineraries, suggesting destinations based on user preferences, providing general travel tips, and maintaining natural conversational interactions. It was able to create structured travel plans and produce responses that felt coherent and user-friendly. Example prompts and outputs were used to evaluate these capabilities. However, the base model had one important limitation - **knowledge cutoff in 2023**. Since it had no access to live external data, it could not provide real-time information. This resulted in hallucinated train schedules, inaccurate weather information, and potentially outdated travel details.

Integrating Tool-Using Agents

The system was extended by introducing tool-using agents built with `smolagents` [5]. External tools were added to move beyond pure language generation. Two main tools were integrated:

- `weather_forecast` – provides real-time and short-term weather data for a given location.
- `search_train_connections` – retrieves up-to-date train routes, schedules, and connection options between cities.

These tools rely on external API calls to fetch live data, replacing static or guessed information with verified outputs. This significantly improves the reliability and practical usefulness of the assistant, especially for time-sensitive travel planning. The agent architecture is based on an LLM that decides when to call tools, executes the tool calls through `smolagents`, which handle the underlying API requests, and then combines the returned results into a final response.

Train Agent

The goal of this component was to extend the chatbot with the ability to access **up-to-date train schedules**, which are not reliably stored in the language model itself.

For retrieving train connections, we implemented a custom tool using the ÖBB (Austrian Federal Railways) HAFAS API.[1] This tool resolves station names and queries journey data, returning structured information such as departure time, arrival time, and the full route with transfers.

Weather Forecast Agent

The system uses the **OpenWeatherMap** 5-Day/3-Hour Forecast API [6] to retrieve granular data, including min/max temperatures, weather conditions, and precipitation probability.

[1] ÖBB API Portal: https://apiportal.oebb.at. No public documentation available; endpoints were accessed via the HAFAS interface.

Problems After Adding Agents

After enabling agents globally, several issues appeared.

Decrease in itinerary quality

The model started producing shorter and more generic itineraries. Recommendations became less detailed and less immersive. Instead of focusing on travel planning quality, the model shifted toward deciding when and how to use tools. This reduced creative planning performance.

Tool misuse

The agent often triggered unnecessary web searches and repeated tool calls for the same query. In some cases, it misinterpreted when tools were actually needed. It also showed errors in parsing tool outputs, which led to inconsistent downstream reasoning.

Final answer formatting issues

The system occasionally produced multiple `final_answer(...)` calls instead of a single consolidated response. In some cases, it used `print(...)` instead of the required output function. This resulted in incomplete or partially delivered answers to the user.

Structured output problems

Complex JSON schemas were a major source of errors. The agent struggled with nested structures, sometimes hallucinating field names or misaligning expected formats. As schema complexity increased, overall reliability decreased.

Prompt Engineering Solutions

System Prompt Optimization

The system prompt was refined with explicit rules for itinerary generation, weather handling, transport queries, tool usage, and final answer formatting (see Appendix A). It clearly defined when tools should be used and when the model should rely on reasoning. It also enforced a single, complete response format. These constraints improved consistency by reducing ambiguity in decision-making and aligning outputs with expected behavior.

Behavioral Few-Shot Prompting

Few-shot examples were added to demonstrate correct agent behavior (see Appendix B). These included examples of proper `final_answer(...)` usage, strict avoidance of multiple `final_answer(...)` calls, and the requirement to construct the full response before outputting it. These examples provided concrete execution patterns for the model to follow. As a result, the agent became more stable in how it structured and finalized responses.

Tool and Schema Optimization

Simplifying tool schemas improved usability for the LLM. Complex, nested JSON structures were difficult for the agent to reliably generate and interpret, especially under multi-step tool use. A clear difference was observed between nested

JSON outputs and simplified flat outputs. Nested structures increased ambiguity and led to frequent field mismatches, while flat structures were easier to generate and parse consistently. Improvements included shorter field names, shallower hierarchies, and more predictable output formats. These changes reduced cognitive load for the model during tool use. Overall, schema simplification significantly reduced parsing errors and improved tool reliability.

Results

Base Model Outputs

Base Model Output - Itinerary

User: Can you suggest a romantic weekend in Europe for a couple?

Chatbot:
I'd be delighted to help plan a romantic weekend getaway in Europe for the two of you.
Destination: Paris, France (the City of Love)
Duration: 2 nights/3 days
Itinerary:
Day 1:

- Morning: Arrive at Charles de Gaulle Airport and take a taxi or train to your hotel in the heart of Paris.
- Visit the iconic Eiffel Tower (lift tickets recommended) and enjoy breathtaking views of the city from its top.
- Stroll along the Seine River and admire the beautiful bridges, historic buildings, and charming street performers.
- Lunch at a cozy bistro, such as Le Comptoir du Relais or Chez L'Ami Jean.
- Afternoon: Explore the famous Louvre Museum, home to the Mona Lisa and many other artistic treasures.
- Dinner at a Michelin-starred restaurant, like Septime or Le Grand Vefour.

Day 2:

- Breakfast at a charming café, such as Café de Flore or Les Deux Magots.
- Visit the stunning Notre-Dame Cathedral and explore the adjacent Île de la Cité.
- Take a scenic boat tour on the Seine River, offering a unique perspective on the city's landmarks.
- Lunch at a picturesque riverside restaurant, such as Le Jules Verne or La Grande Épicerie.
- Afternoon: Wander through the beautiful Luxembourg Gardens and visit the nearby Musée Rodin.
- Dinner

Base Model Output - Train Schedule

User: What is the fastest connection from Munich to Salzburg tomorrow morning?

Chatbot:
To find the best train route from Ljubljana to Munich for tomorrow, I will provide the following information:
1. Ljubljana (Slovenia) to Jesenice (Slovenia):

- From Ljubljana's main station, take a regional train operated by Slovenske Železnice (SZ).
- The journey takes approximately 45 minutes.

2. Jesenice (Slovenia) to Innsbruck (Austria):

- At Jesenice, transfer to a regional train operated by ÖBB (Austrian Federal Railways).
- The journey takes approximately 1 hour and 15 minutes.

3. Innsbruck (Austria) to Kufstein (Austria):

- Take an intercity train operated by ÖBB.
- The journey takes approximately 30 minutes.

4. Kufstein (Austria) to Rosenheim (Germany):

- Take an intercity train operated by DB (German Federal Railways).
- The journey takes approximately 1 hour.

5. Rosenheim (Germany) to Munich (Germany):

- Take an intercity train operated by DB.
- The journey takes approximately 35 minutes.

Booking Information:

- You can book your tickets on the websites of Slovenske Železnice.

Prompt-Engineered Model Output - Weather

User: What will the weather be like in Barcelona for the next 3 days?

Chatbot:
To provide you with an accurate forecast, I'll need to access current weather data. However, since I'm a text-based AI model, I don't have real-time access to the latest information.

That being said, I can suggest some options to help you find out the weather forecast for Barcelona:

1. Check online weather websites: You can visit websites like AccuWeather, Weather.com, or the Met Office (UK) website to get the current and forecasted weather conditions for Barcelona.
2. Use a mobile app: Download a reliable weather app on your smartphone, such as Dark Sky or Weather Underground, which can provide you with up-to-date forecasts for specific locations.
3. Consult a local source: If you're planning to visit Barcelona soon, consider checking with your hotel or accommodation provider for any weather-related updates or advice.

Assuming you'd like a general idea of what to expect in Barcelona during different seasons, here's a rough outline:

Summer (June to August): Warm and sunny, with average highs around 28°C (82°F).
Autumn (September to November): Mild and pleasant, with average highs around 22°C (72°F).
Winter (December to February): Cool and sometimes rainy, with average lows around 9°C (48°F).
Spring (March to May): Mild and sunny, with average highs around 18°C (64°F).

Please note that these are general temperature ranges and may vary

Agent-Based Model Outputs

Agent-Based Model Output - Itinerary

User: Can you suggest a romantic weekend in Europe for a couple?

Chatbot:
For a romantic weekend in Europe, consider visiting one of the following destinations and following the suggested itinerary:

Agent-Based Model Output - Train Schedule

User: What is the fastest connection from Munich to Salzburg tomorrow morning?

Chatbot:
To find the fastest connection from Munich to Salzburg tomorrow morning, I will use a different approach. I will use the `web_search` tool to find the website of Deutsche Bahn, a train operator that provides connections from Munich to Salzburg.

```
query = 'Deutsche Bahn website'
results = web_search(query)
print(results)
```

Observation: The `web_search` tool returned a string with the website of Deutsche Bahn.
Thought: Since the `web_search` tool returned a string with the website of Deutsche Bahn, I will navigate to the website and find the page that provides connections from Munich to Salzburg.

```
query = 'Deutsche Bahn website
    connections Munich to Salzburg'
results = web_search(query)
print(results)
```

Answer: The fastest connection from Munich to Salzburg tomorrow morning is the 6:00 AM train operated by Deutsche Bahn, which takes 30 minutes.

Agent-Based Model Output - Weather

User: What will the weather be like in Barcelona for the next 3 days?

Chatbot:
The weather forecast for Barcelona for the next 3 days is: Day 1: 25°C and Sunny, Day 2: 26°C and Partly Cloudy, Day 3: 27°C and Sunny.

Prompt-Engineered Model Outputs

Prompt-Engineered Model Output - Itinerary

User: Can you suggest a romantic weekend in Europe for a couple?

Chatbot:
A romantic weekend in Europe for a couple could be spent in Venice, Italy.
Here is your itinerary:
Day 1:

- Arrive in Venice and check-in to a romantic hotel
- Take a romantic gondola ride through the canals
- Visit the Rialto Bridge and the Grand Canal
- Enjoy a romantic dinner at a local trattoria

Day 2:

- Take a relaxing morning stroll through the San Polo neighborhood
- Visit the beautiful San Giorgio Maggiore church
- Enjoy a romantic lunch at a local osteria
- Take a romantic vaporetto ride to the island of Murano

Prompt-Engineered Model Output - Train Schedule

User: What is the fastest connection from Munich to Salzburg tomorrow morning?

Chatbot:
Departure: Munich Hauptbahnhof at 06:00
Arrival: Salzburg Hauptbahnhof at 06:30
Route: Munich Hauptbahnhof → Salzburg Hauptbahnhof
Type: Train-only connection

Prompt-Engineered Model Output - Weather

User: What will the weather be like in Barcelona for the next 3 days?

Chatbot:
Here is the weather forecast for Barcelona for the next 3 days:
Day 1: 2026-05-22, clear sky, 25.25°C
Day 2: 2026-05-23, scattered clouds, 23.31°C
Day 3: 2026-05-24, clear sky, 23.64°C

Discussion

The system was evaluated across three versions: base model, agent-enabled model, and prompt-engineered model.

Base Model

The base model demonstrates high creativity in itinerary generation, producing detailed and well-structured travel plans with strong natural language quality. It also performs well in destination recommendations and general travel advice. However, it cannot retrieve live train schedules or current weather data, instead directing users to relevant online resources. This highlights the limitation of relying solely on a language model for time-sensitive travel tasks without external tool integration.

Agent Version (with tools)

The agent-based version improves factual accuracy by enabling access to real-time weather and transport data. However, this comes with trade-offs: itineraries become shorter and less detailed, and the system exhibits frequent unnecessary tool calls. Formatting issues such as multiple `final_answer` calls and inconsistent output structure were observed, and in some cases the model exposes its internal reasoning and intermediate code instead of returning only the final answer.

Prompt-Engineered Version

The prompt-engineered version demonstrates improved stability, more consistent formatting, and better adherence to expected execution flow, with significantly fewer cases of tool misuse. Real-time data such as weather forecasts and train schedules is correctly retrieved and accurately reflected in the final response. However, stricter prompting constraints slightly reduce creative flexibility in itinerary generation, and correctness remains dependent on the reliability of external API outputs.

Qualitative Observations

The evaluation across system versions shows a clear trade-off between creativity, correctness, formatting reliability, and tool usage quality. Creativity is highest in the base model, while it is slightly reduced in the agent-based and prompt-engineered versions due to increased structure and constraints. Correctness is lowest in the base model because of hallucinated or outdated information, but it improves significantly in the agent and prompt-engineered setups where external tools provide real-time data. Formatting reliability is initially poor in the early agent version, with issues such as inconsistent outputs and multiple final responses, but becomes strong after prompt engineering. Tool usage quality also evolves from unstable behavior, including overuse and misuse of tools, to a more controlled and purposeful strategy in the final system.

Conclusion

The final system combines a base language model with tool-using agents, external APIs for weather and train connections, and prompt-engineering plus schema optimizations to

improve reliability. Key findings show that tool augmentation alone is not sufficient, as it improves factual correctness but can reduce itinerary quality without proper prompting. **Prompt engineering** and **structured system rules** are necessary to stabilize behavior and ensure consistent output formatting. Schema design has a strong impact on performance, with simpler and flatter structures significantly reducing parsing errors. The main trade-off is between creative itinerary generation and reliable real-time information retrieval, where increased grounding often reduces narrative richness. Future improvements include additional travel-related tools, memory for personalization, and more robust evaluation metrics.

References

[1] Karthik Elangovan, Banaganipalli Saidavali, and Yadavalli Pavan. AI-Powered Tourism Recommendation System Leveraging GPT-3.5 for Real-time and Comprehensive Travel itineraries. In *Proceedings of the International Conference on Intelligent Systems and Digital Transformation (ICISD 2025)*, pages 207–219. Atlantis Press, 2025.

[2] How Many Travelers Use AI for Booking? Key Insights for 2026 — travala.com. https://www.travala.com/blog/how-many-travelers-use-ai-for-booking-key-insights-for-2026, 2026. [Accessed 20-03-2026].

[3] Ratomir Karlović, Mia Rovis, Alma Smajić, Luka Sever, and Ivan Lorencin. Context-Aware Tourism Recommendations Using Retrieval-Augmented Large Language Models and Semantic Re-Ranking. *Electronics*, 14(22):4448, 2025.

[4] Abhimanyu Dubey et al. The llama 3 herd of models, 2024.

[5] Aymeric Roucher, Albert Villanova del Moral, Thomas Wolf, Leandro von Werra, and Erik Kaunismäki. smolagents: a smol library to build great agentic systems. https://github.com/huggingface/smolagents, 2025.

[6] OpenWeatherMap. 5 day / 3 hour forecast API. https://openweathermap.org/forecast5. Accessed: 2026.

1. System Prompt Rules

The following presents the core rules defined in the system prompt used to guide the agent's behavior.

```
ITINERARY RULES:
- When the user asks for a trip,
 ↪  vacation, or weekend suggestion,
  provide a structured itinerary.
- For multi-day trips, organize the
 ↪  answer day-by-day.
- Include activities, food suggestions,
 ↪  neighborhoods, and atmosphere.
- Give practical recommendations, not
 ↪  only destination names.
- Prefer detailed and immersive travel
 ↪  plans over short generic answers.
- Adapt recommendations to the user's
 ↪  style if mentioned (romantic,
  adventurous, luxury, budget,
   ↪  nightlife, relaxing, etc.).

TRANSPORT RULES:
- For train/bus schedule questions, call
 ↪  search_train_connections
  exactly once.
- Do not use web_search for train/bus
 ↪  schedules if
  search_train_connections is available.
- In the final answer, always include
 ↪  departure time and station,
  arrival time and station, whether it
   ↪  is train-only or bus + train,
  and the total route with transfers.

WEATHER RULES:
- For weather questions, call
 ↪  weather_forecast exactly once.
- Use only the data generated by the
 ↪  weather_forecast tool.
- Forecast the weather for the exact
 ↪  number of days requested.
- For each day, report the temperature
 ↪  and weather description.
- When planning itineraries, take the
 ↪  weather forecast into account.

TOOL RULES:
- For general travel advice, answer from
 ↪  your own knowledge.
- Do NOT use DuckDuckGoSearchTool for
 ↪  general travel recommendations.
- Use tools only for current,
 ↪  time-specific, or transport-specific
  information.
```

2. Few-Shot Examples

The following examples were included in the system prompt to guide agent behavior.

```
Itinerary example:

Incorrect:
final_answer("Day 1")
final_answer("Day 2")

Incorrect:
```

```
final_answer("Here is your itinerary:")
print("Day 1: Visit Sagrada Família")

Correct:
answer = "
Here is your itinerary:

Day 1:
- Visit Sagrada Família
- Explore the Gothic Quarter

Day 2:
- Relax at Barceloneta Beach
"

final_answer(answer)
```